

Coding & Solution

Date	02 October 2026
Team ID	SWTID-2026-1380
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/9993082160-rgb/Kaviya.git
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	FastAPI (served with Uvicorn), Jinja2 templates, Pydantic; Google Generative AI SDK (Gemini 1.5 Flash Pro), passlib (bcrypt) and python-jose (JWT), Pillow
Key Features Implemented	• Three budget planners: Home Interior, Party and Jewelry • Gemini-generated recommendations returned as structured JSON with a budget breakdown, calculation table and remaining budget • Product and service search links for platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO • Jewelry planner with optional outfit image upload for style-matched suggestions • User registration, login and logout with bcrypt password hashing and JWT-based sessions • User dashboard and recommendation history page • Home page, testimonials page and responsive HTML/CSS interface

Pending / Incomplete Features	• Split app.py into separate routes, services and models modules • Share the repeated shopping-link code across the three planners • Restrict CORS to specific origins for production • Add automated tests • Prices are AI estimates and links are search links; live product and price data is not yet integrated
Setup / Run Instructions	1. Clone the repository and open the project folder. 2. Create and activate a virtual environment: <code>python -m venv venv</code> 3. Install the dependencies: <code>pip install fastapi uvicorn jinja2 python-multipart pydantic[email] passlib[bcrypt] python-jose pillow google-generativeai python-dotenv</code> 4. Create a <code>.env</code> file in the project folder with your Gemini API key: <code>GOOGLE_API_KEY=your_api_key</code> 5. Start the server: <code>uvicorn app:app --reload</code> 6. Open <code>http://127.0.0.1:8000</code> in a browser, register an account and log in.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The three planners follow the project description and workflow provided by the lecturer. Gemini 1.5 Flash Pro generates the recommendations, and the FastAPI backend handles routing, login and sessions. The API key is kept in the .env file and must not be uploaded to GitHub.